

## 第3方案实现文档

混合架构: `Lianjia-beike-spider` 做 “小区价格底座 (周/月) ” ,  
`xjkj123/Lianjia` 做 “房源优缺点评分引擎 (可解释) ” 。  
Web: `Vue + ECharts` , 支持多城市 (先深圳跑通) 。  
时间粒度: 按周为主 (默认周) , 可切换月。

### 0. 已确认的 3 个决策 (本版落地)

1. `weekly` 定义: 从抓取日期按 7 天窗口定义
  - 对抓取落库日期 `D` , 周窗口为 `[D-6, D]` (`week_start=D-6`, `week_end=D`)
2. 学校筛选/分级与 “未来走向”
  - 学校分级筛选包含 “省一级 / 市一级” 维度 (作为硬筛选标签之一)
  - 同时输出 “未来走向趋势分” (融合最新办学水平、集团化办学、优质均衡区县等指标, 规则可解释)
3. 均价取策略
  - 默认: 窗口内 “最新非空” 口径 (尽量贴近开源仓库随时间滚动采样的行为)
  - 兜底: 若最新非空不足或异常, 切换到 “中位数 `median`” 做稳健聚合

### 1. 技术选型 (可直接开工)

1. 前端
  - `Vue 3 + Vite + TypeScript`
  - `ECharts 5`
  - `Element Plus` (表单/表格/筛选 UI)
  - `Pinia` (状态管理)
2. 后端
  - `FastAPI`
  - `SQLAlchemy 2.x`
  - `Pydantic` (请求/响应校验)

### 3. 数据库与缓存

- PostgreSQL 15+ (推荐 JSONB 存半结构字段与解释信息)
- Redis (建议, 用于缓存聚合结果、排行榜、常用口径结果)

### 4. ETL / 评分与调度

- Python 3.11
- pandas/numpy (清洗与聚合)
- rapidfuzz (中文模糊匹配: 学校/社区别名)
- 定时: APScheduler (MVP), 后期可迁移 Airflow (规模化)

### 5. 爬虫集成方式

- 不推翻开源仓库: 用 Adapter 将各来源输出统一到我们的 schema

---

## 2. 项目工程结构 (建议目录)

(后端)

```
housing-price-qa/  
  backend/  
    app/  
      api/v1/  
        communities.py  
        listings.py  
        schools.py  
        stats.py  
      services/  
        community_service.py  
        school_service.py  
        scoring_engine.py  
        snapshot_service.py  
      etl/  
        connectors/  
          official_public.py  
          lianjia_xiaoqu_adapter.py  
          lianjia_listing_adapter.py  
        normalize/  
          community_resolution.py  
          school_standardize.py  
        features/  
          school_future_features.py  
          listing_feature_extract.py  
        quality/  
          dedupe_rules.py  
          coverage_checks.py  
        scheduled/  
          weekly_snapshot_job.py  
          monthly_snapshot_job.py  
          scoring_job.py  
      models/  
        schema.py  
      main.py  
      requirements.txt  
      Dockerfile
```

(前端)

```
frontend/  
  src/  
    api/  
    components/  
      charts/  
        PriceTrendChart.vue  
        RankingChart.vue  
        RadarQualityChart.vue  
        ScoreDistributionChart.vue  
      filters/  
        CitySelect.vue  
        PeriodSwitch.vue  
        ListingFilters.vue  
      table/  
        CommunityRankingTable.vue  
        ListingTable.vue  
    pages/  
      Dashboard.vue  
      CommunityDetail.vue  
      ListingSearch.vue  
  package.json  
  Dockerfile
```

(脚本/部署)

```
scripts/  
  init_db.sql  
  docker-compose.yml
```

### 3. 数据模型（最小可用但可扩展）

#### 1. 城市与区县

- city
- district （可选，用于聚合与“优质均衡区县”指标占位）

#### 2. 社区实体（多源对齐关键）

- community\_official：统一口径的小区实体（作为主键实体）
- community\_mapping：source 小区名 -> community\_official 映射表（带 match\_score、match\_rule、human\_verified）

#### 3. 学校标准化与分级（你要的“历史名校/强学区 + 未来走向”）

- school\_standardized

- 标准学校实体: `official_name`、`display_name`、`school_type`、`province_key_flag` (省一级)、`city_key_flag` (市一级) 等
- `school_name_alias`
  - 别名表: 同校变体/简称/错别字 -> 标准学校
- `school_indicator_snapshot`
  - 指标快照表: 用于承载“最新办学水平、集团化、优质均衡区县”等输入指标 (按日期/来源留痕)
- `school_future_score`
  - 未来走向趋势分: `trend_score_0_100` + `feature_contrib_json` / `explain_text` (可解释)

#### 4. 小区价格快照 (周/月)

- `price_snapshot_weekly`
- `price_snapshot_monthly`
- 两者都建议包含: `avg_unit_price`、`median_unit_price` (可选)、`listing_count`、`data_policy`、`source_priority_used`、覆盖/缺失的置信指标

#### 5. 房源明细 (用于优缺点评分)

- `listing_detail`
- 建议保留: 价格字段 (总价/单价/面积)、房龄/朝向/楼层/电梯/装修类型、学校关联 (先存 `school` 名或学校列表, ETL 后再落到 `standardized_school_id`)、地铁距离结构化字段 (如果来源能提取结构化)

#### 6. 房源优缺点评分 (可解释输出)

- `listing_quality_score`
- 建议包含: `rule_version`、`overall_score_0_100`、各维度分、`advantages_json`、`disadvantages_json`、`explain_json` (规则命中、阈值、贡献分等)

## 4. `weekly` 快照生成算法 (抓取日期按 7 天窗口)

### 1. 窗口定义

- 对抓取落库日期 `D`:
  - `week_start = D - 6 days`
  - `week_end = D`
- 对每个 `community_id` 在该窗口内生成一条 `price_snapshot_weekly`

### 2. 均价口径 (默认可切换)

- `latest_non_null` (默认) : 窗口内按抓取日期取 “最新且非空” 的 `unit_price` 作为 `avg_unit_price`
- `median_robust` (兜底) : 若最新非空不足或数据明显异常, 则取窗口内 `unit_price` 的 `median`

### 3. 覆盖率与缺失标记 (前端必须能感知)

- `coverage_score = min(1.0, sample_count / target_listing_count)`
- 若 `coverage_score` 低于阈值, 前端展示 “置信度低/缺失风险”, 避免用户误判走势

### 4. 建议默认参数 (MVP)

- `min_samples_for_latest = 5`
- `target_listing_count = 30`
- `score_policy_default = latest_non_null` , 兜底到 `median_robust`

---

## 5. 学校名称标准化 + 分级 + 未来走向趋势分 (可解释、可追溯)

### 1. 学校名称标准化 (实体对齐)

- 清洗原始学校名 (去空格、统一后缀, 如 “学校/校/分校/附中/附小” 等映射)
- 优先命中 `school_name_alias`
- 再做同城模糊匹配 (用 `rapidfuzz / token similarity`)
- 匹配置信度过低的进入后台审核队列 (必须可控、可复盘)

### 2. 分级筛选 (省一级 / 市一级)

- `province_key_flag` 、 `city_key_flag`
- 数据来源优先官方/公开平台; 若缺失则标注口径缺失并在前端以 `unknown` 方式处理 (避免 “假精确” )

### 3. 未来走向趋势分 (Rule-based 输出, 带解释)

- 输出: `trend_score_0_100`
- 输入特征集合 (MVP 可配置, 先保证能算)
  - `latest_level_score` : 最新办学水平量化分 (缺失时回退)
  - `group_school_bonus` : 集团化办学加分
  - `district_balance_bonus` : 优质均衡区县加分 (按你接入的区县指标档位)
  - `trend_delta_score` : 近 1-2 期变化趋势 (上升/下降幅度)
- 默认加权 (可版本化)
  - `trend_score = 0.55 * latest + 0.20 * group + 0.15 * district + 0.10 * delta`

- 可解释存储要求
  - 落库 `feature_contrib_json` 和 `explain_text` , 前端可展示 “为何高/为何低”

#### 4. 规则版本管理

- `school_future_score` 必须带 `rule_version` (后续迭代规则时不会破坏历史可复现)

---

## 6. 房源优缺点评分引擎 (规则引擎 + `explain_json` + 版本化)

### 1. 评分维度 (默认 5 维)

- `location_score` : 地铁距离 + 学校 (用学校未来趋势分贡献)
- `house_quality_score` : 朝向 + 户型适配
- `building_age_score` : 房龄衰减 (`build_year`)
- `amenity_score` : 电梯/装修类型
- `price_value_score` : 相对小区价格快照的性价比

### 2. 默认阈值 (MVP 直接落地)

- 地铁距离 (如果有 `nearest_metro_distance_m`) :
  - `<500m` 高
  - `500-1000m` 中
  - `1000-2000m` 低
  - `>2000m` 很低
- 学校贡献:
  - 房源关联学校列表中取 `max(school_future_score)` 作为学校质量代表值 (MVP 默认 `max`; 后续可切换 `avg`)
- 房龄衰减 (按建成年份换算)
  - `<5年` 高
  - `5-10` 中高
  - `10-15` 中
  - `15-20` 中低
  - `>20` 低
- 电梯/装修:
  - 有电梯加分更高
  - 精装/简装/毛坯按映射表给分 (阈值后续可调)
- 性价比:
  - `ratio = listing_unit_price / community_avg_unit_price_weekly`

- `ratio < 0.90` 性价比高得分更高
- `ratio > 1.15` 得分更低

### 3. 优点/缺点标签生成

- `advantages_json` : 命中正向规则的标签集合
- `disadvantages_json` : 命中负向规则的标签集合
- 每个标签带 `confidence` (来自规则命中等级/固定置信)

### 4. explain\_json 结构约束 (保证 “可解释” )

- 包含: `rule_version` 、每维度的命中阈值区间、用到的输入字段名、维度贡献分、最终 `overall_score`

---

## 7. 后端 API 设计 (为前端图表/筛选服务)

### 1. 城市与期别

- `GET /api/v1/cities`
- `GET /api/v1/periods?type=weekly|monthly&cityId=...`

### 2. Dashboard 聚合 (图表用)

- `GET /api/v1/stats/community-price-trend?cityId&startWeekEnd&endWeekEnd&periodType=weekly|monthly`
- `GET /api/v1/stats/community-ranking?cityId&periodType=weekly&weekEnd=YYYY-MM-DD&metric=avg_unit_price|listing_count`
- `GET /api/v1/stats/district-compare?cityId&periodType=weekly&weekEnd=...` (区/板块对比)

### 3. 小区详情页

- `GET /api/v1/communities/{communityId}/price-trend?...`
- `GET /api/v1/communities/{communityId}/quality-summary?days=30`
- `GET /api/v1/communities/{communityId}/top-tags?limit=20`

### 4. 房源筛选 (优缺点分析核心)

- `POST /api/v1/listings/filter`
- 支持筛选维度: 价格区间、面积区间、房龄、电梯、装修、朝向、学校分级 (省一级/市一级)、学校未来趋势阈值、质量分范围
- 排序: `overall_score desc` 、 `price_value desc` 、 `latest_price asc/desc` 等
- 分页: `page/pageSize` (保证前端可用)

### 5. 学校搜索与筛选 (给前端联动)

- `GET /api/v1/schools/search?cityId&q=...`



- GET /api/v1/schools/{schoolId}/future-score

## 8. 前端页面落地 (Vue + ECharts, 参考你的“顶部筛选 + 多图表卡片 + 详情联动”风格)

### 1. Dashboard (MVP)

- 顶部: 城市选择 + 周/月切换 + 时间范围 + (可选) 关键筛选项
- 图表卡片:
  - 全城价格趋势 (折线)
  - 区/板块对比 (柱状或热力)
  - 小区排行 (Top N 表格为主, 图表为辅)
- 点击排行行: 跳转 `CommunityDetail`

### 2. CommunityDetail

- 价格走势折线 (周/月)
- 评分分布图 (直方图/饼图: 按 `overall_score` 分箱)
- 优缺点评价可视化:
  - 雷达图 (维度固定, 值来自 `dimension_scores_json`)
  - 标签分布 (top tags)
- 房源列表 (表格 + 条件筛选联动)

### 3. ListingSearch (可选但建议)

- 以“学校强度 (省/市级 + 未来趋势分) + 房源质量分”作为主要筛选入口
- 更贴近你“优缺点分析”的核心使用场景

### 4. 图表数据契约 (必须提前定义)

- 折线图: `[{x: period_start/week_end, y: avg_unit_price, missingFlag}]`
- 雷达图: 维度固定数组 + values 来自 `dimension_scores_json`
- 分布图: 分箱区间 + counts 来自后端聚合

## 9. ETL 与调度 (保证按周/月稳定出数)

### 1. MVP 节奏 (推荐)

- 每天更新房源明细 `listing_detail` (来自 `xjkj123/Lianjia` 适配器)
- 每次抓取落库后刷新受影响小区的相关数据 (可选缓存刷新)
- 定时生成周快照 `price_snapshot_weekly` (滚动 7 天窗口口径)
- 当学校指标快照更新时重算 `school_future_score`

- 当 `listing_detail` / 小区快照 / 学校未来分变化时重算 `listing_quality_score`

## 2. 幂等性要求（必须写进实现）

- 周快照唯一键建议： `(community_id, week_end_date)` （避免重复写）
- 房源评分唯一键建议： `(listing_id, rule_version)` （支持规则版本回放）

## 3. 调度方式（MVP）

- `APScheduler` 做定时任务（周/日/触发式都可）
- 后续数据量变大再迁移 `Airflow`

---

# 10. 数据质量检查（没有它就很难“可信可视化”）

## 1. 覆盖率与样本量

- `listing_count` 、 `coverage_score`
- 明确标注“低覆盖周”的置信度

## 2. 缺失率与字段完整度

- `unit_price` 缺失占比
- 学校字段缺失占比
- 学校匹配置信度分布（高置信/低置信占比）

## 3. 异常检测（防止图表“突变误导”）

- 与上一次相邻窗口的均价偏差是否超过阈值（例如 `>30%` 标记异常）
- 前端显示“异常/缺失周”的标注信息（例如加红点/灰线缺口）

## 4. 规则与解释一致性

- `listing_quality_score` 必须记录 `rule_version`
- `explain_json` 必须能回溯到实际输入字段与命中阈值区间